



Scenes System: SceneLoader, DuplicateBag, FadeManager, LevelManager

SceneLoader.

В Unity есть SceneManager.LoadScene(), который позволяет загрузить нужную сцену. Однако это простой способ, которого мне недостаточно. Мне нужна надстройка над загрузкой сцены.

Во-первых, стоит сразу сказать, что сцену мы будем загружать аддитивно:

LoadSceneMode.Additive, - поверх той, что у нас есть сейчас. Сделано это для того, чтобы во всех случаях, всех уровнях и меню, фоново всегда жила сцена Boot, которую было правильнее назвать Persistent (постоянная). В ней живут все singleton менеджеры, HUD, Settings-Menu, Prestart-Screen, Death-Screen.

И так... Вызвать загрузку уровня мы можем через LoadScene() передав туда либо название string формата, либо тип данных LevelEntry, который также хранит название сцены. После чего мы запустим корутину LoadSceneRoutine.

Начальная версия загрузчика была в разы проще, но позже были исправлены различные ошибки и расширен функционал. В процессе разработки было замечено, что игровой уровень подгружался дважды, из-за чего вся логика игры сыпалась. Сейчас мы контролируем загрузку через isLoading флаг, а также через прямой контроль корутины. Две проверки могут быть излишне, но они нужны были в процессе поиска решения бага.

Когда мы начинаем загрузку, то устанавливаем флаг и присваиваем в поле корутину. К слову, теперь мы передаём callback'и в виде onLoaded & onCompleteFade. Первый - для контроля загрузки сцены, а второй - для контроля, когда закончится затемнение и экран полностью окажется скрыт.

```
1 using System;
2 using System.Collections;
3 using UnityEngine;
4 using UnityEngine.SceneManagement;
5
6 public class SceneLoader : Manager<SceneLoader>
7 {
8     private Scene currentScene;
9     private bool isLoading = false;
10    private Coroutine loadRoutine;
11
12    public void LoadScene(string newSceneName, Action onLoaded = null,
13    {
14        if (isLoading) return;
15        if (loadRoutine != null)
16        {
17            return;
18        }
```

```

19
20     isLoading = true;
21
22     loadRoutine = StartCoroutine(LoadSceneRoutine(newSceneName, onL
23 }
24
25 public void LoadScene(LevelEntry levelEntry, Action onLoaded = null
26 {
27     if (isLoading) return;
28     if (loadRoutine != null)
29     {
30         return;
31     }
32
33     isLoading = true;
34
35     loadRoutine = StartCoroutine(LoadSceneRoutine(levelEntry.sceneN
36 }
37
38 private IEnumerator LoadSceneRoutine(string sceneName, Action onLoa
39 {
40     bool fadeDone = false;
41     FadeManager.FadeOutThen(() => fadeDone = true, 1.5f);
42     yield return new WaitUntil(() => fadeDone);
43
44     if (currentScene.IsValid())
45     {
46         yield return SceneManager.UnloadSceneAsync(currentScene);
47         currentScene = default;
48     }
49
50     AsyncOperation loadOp =
51         SceneManager.LoadSceneAsync(sceneName, LoadSceneMode.Additi
52     yield return loadOp;
53
54     Scene newScene = SceneManager.GetSceneAt(SceneManager.sceneCoun
55     SceneManager.SetActiveScene(newScene);
56     currentScene = newScene;
57
58
59     FadeManager.FadeInThen(
60         () => { onCompleteFade?.Invoke(); },
61         1.5f
62     );
63
64     onLoaded?.Invoke();
65
66     isLoading = false;
67     loadRoutine = null;
68 }
69
70 public void SetCurrentScene(Scene scene)
71 {
72     currentScene = scene;
73 }
74 }

```

LoadSceneRoutine. Мы устанавливаем флаг затемнения - выключен. И вызываем из FadeManager начать затемнение, куда передаём флаг, который станет true, и время перехода. Сразу же начинаем ожидать через WaitUntil() этого момента.

После мы проверяем текущую сцену на валидность (сцена реально лежит в поле). Если всё нормально, то запускаем асинхронную выгрузку сцены. А в текущую сцену ложим

default.

К слову, Boot сцена никогда не попадёт в currentScene поле, потому что она запускается первой и вызывает через отдельный одноразовый скрипт загрузку MainMenu. Скрипт будет ниже.

Мы создаём AsyncOperation и присваиваем в него загрузку новой сцены, после чего ожидаем загрузку. Делаем новую сцену активной, и указываем её как текущую.

Запускаем “растемнение” экрана, передаём калбек onCompleteFade ещё глубже в менеджер затемнения. Там будет обрабатывать короутина, которая в конце оповестит подписчиков события.

Сразу же вызываем подписчиков onLoaded, выключаем флаг и очищаем короутину. Когда растемнение экрана закончится - вызовутся подписчики onCompleteFade. На этом загрузка сцены завершена . . .

```
1 private void LoadStage()
2 {
3     if (State == GameState.Loading) return;
4     State = GameState.Loading;
5
6     AudioManager.Instance.StopMusic();
7
8     SceneLoader.Instance.LoadScene(
9         LevelManager.Instance.GetRandomLevel(),
10        () =>
11        {
12            restartLocked = false;
13
14            DifficultyManager.Instance.SetFloor(GameManager.Instance.Fl
15            WalletManager.Instance.ResetWallet();
16            BaseManager.Instance.ResetBase();
17            HudManager.Instance.Reset();
18            HudManager.Instance.Show();
19            State = GameState.Prestart;
20            EnemySpawnerManager.Instance.StopSpawning();
21            EnemySpawnerManager.Instance.SceneEnabled(5);
22        },
23        () => { HudManager.Instance.ShowPrestartButton(); }
24    );
25 }
26 // GameFlowManager.cs
```

Таким образом вызывается данный загрузчик сцен. Мы передаём название сцены, которое получим из LevelManager, а также передаём лямбдой анонимный метод, который будет содержать нужный нам функционал. В первом случае всё, что произойдёт после загрузки сцены. Во втором случае - мы покажем кнопку старта игры.

Баг дублирования.

Я уже упоминал выше, что столкнулся с багом дублируемой загрузки игровых сцен. Времени было потрачено много.

Во-первых, в GameFlowManager (управляющим циклом игры) пришлось сделать контроль состояний:

```
1 public enum GameState
2 {
3     Menu,
4     Loading,
5     Playing,
6     ArtifactChoice,
7     Defeat,
8     Prestart,
9     EscMenu
10 }
11
12 private void LoadStage()
13 {
14     if (State == GameState.Loading) return;
15     State = GameState.Loading;
16     . . .
17 }
18
19 public void StartPlaying()
20 {
21     if (State != GameState.Prestart) return;
22     State = GameState.Playing;
23     . . .
24 }
25
26 ...
27 // GameFlowManager.cs
```

Это было сделано для того, чтобы избежать повторного вызова состояний игры. Ожидалось, что это тоже могло влиять на дублирование сцен.

Во-вторых, во многих менеджерах были добавлены флаги проверок, которые делают запрос к флагу State из GFM. Прежний скрипт загрузки сцены насчитывал меньшее количество флагов и короутины вызывались без присваиваний null и контроля переменной.

Решение оказалось достаточно примитивным. Изначально последовательность загрузки выглядела так: загружаем сцену, выгружаем активную, делаем активной загруженную. Выходило так, что сцена загружалась дважды, и даже если одна из них выгружалась - другая обязательно оставалась. Бывало, что сцена загружалась в единственном экземпляре, но в таком случае она гарантированно не успевала выгрузиться.

Решение - сначала выгружаем текущую сцену, оставляя только фоновый Persistent, затем загружаем уровень и делаем его активным. В коде работоспособность этого

решения выглядела более странно, оно работает.

Код с дублированием сцен:

```
1 using System;
2 using System.Collections;
3 using UnityEngine;
4 using UnityEngine.SceneManagement;
5
6 public class SceneLoader : Manager<SceneLoader>
7 {
8     private Scene currentScene;
9     private bool isLoading = false;
10    private int loadCounter = 0;
11
12    private Coroutine loadRoutine;
13
14    public void LoadScene(string sceneName, Action onLoaded = null)
15    {
16        if (isLoading) return;
17        if (loadRoutine != null)
18        {
19            Debug.LogWarning("[SceneLoader] LoadScene ignored - already loa
20                return;
21        }
22
23        isLoading = true;
24
25        Debug.Log($"[SceneLoader] LoadScene calle: {sceneName}, isLoading={
26        loadRoutine = StartCoroutine(LoadSceneRoutine(sceneName, onLoaded))
27        //StartCoroutine(LoadSceneRoutine(sceneName, onLoaded));
28    }
29
30    public void LoadScene(LevelEntry levelEntry, Action onLoaded = null)
31    {
32        if (isLoading) return;
33        if (loadRoutine != null)
34        {
35            Debug.LogWarning("[SceneLoader] LoadScene ignored - already loa
36                return;
37        }
38
39        isLoading = true;
40
41        Debug.Log($"[SceneLoader] LoadScene calle: {levelEntry}, isLoading=
42        loadRoutine = StartCoroutine(LoadSceneRoutine(levelEntry.sceneName,
43        //StartCoroutine(LoadSceneRoutine(levelEntry.sceneName, onLoaded));
44    }
45
46    private IEnumerator LoadSceneRoutine(string sceneName, Action onLoaded
47    {
48        loadCounter++;
49        int myId = loadCounter;
50
51        Debug.Log($"[SceneLoader #{myId}] START coroutine, frame={Time.frame
52
53        if (isLoading)
54        {
55            Debug.LogError($"[SceneLoader #{myId}] 🚨 ENTERED coroutine whi
56        }
57
58
59        bool fadeDone = false;
60        FadeManager.FadeOutThen(() => fadeDone = true, 2f);
61        yield return new WaitUntil(() => fadeDone);
```

```

62
63     Debug.Log($"[SceneLoader #{myId}] Fade done");
64
65     AsyncOperation loadOp = SceneManager.LoadSceneAsync(sceneName, Load
66     yield return loadOp;
67
68     Debug.Log($"[SceneLoader #{myId}] Scene loaded");
69
70     Scene newScene = SceneManager.GetSceneByName(sceneName);
71     SceneManager.SetActiveScene(newScene);
72
73     if (currentScene.IsValid())
74     {
75         Debug.Log($"[SceneLoader #{myId}] Unloading {currentScene.name}");
76         yield return SceneManager.UnloadSceneAsync(currentScene);
77     }
78
79     currentScene = newScene;
80
81     FadeManager.FadeIn(2f);
82
83     Debug.Log($"[SceneLoader #{myId}] END coroutine");
84     onLoaded?.Invoke();
85     isLoading = false;
86     loadRoutine = null;
87 }
88
89 public void SetCurrentScene(Scene scene)
90 {
91     currentScene = scene;
92 }
93 }

```

BootSceneLoader.

i Одноразовый скрипт отображения загрузочного экрана и загрузка MainMenu сцены.

Начинаем асинхронную загрузку стартовой сцены, в нашем случае - Main_Menu. Пока загрузка идёт вычисляем положение слайдера.

sceneProgress содержит текущий прогресс загрузки сцены. Операция загрузки возвращает 0.0 - 0.9 (90%) - как результат считывания сцены с диска и загрузки в оперативную память. Вот только мы так и не получим результат в виде 1, пока не активируем сцену вручную. А автоматическую активацию мы выключаем - `op.allowSceneActivation = false`; Это сделано на случай, если сцена загрузится слишком быстро, а у нас есть ещё один прогресс. Когда мы поделим `0.9\0.9`, то получим 1.

timerProgress через деление `time \ minLoadTime` переводит секунды в проценты загрузки:

$1\backslash1.5 = 0.6$ $1.5\backslash1.5 = 1$ $2\backslash1.5 = 1.3$

Слайдер работает с процентами в виде 0-1. Поэтому 1.3 = выходу за границы. С помощью `Mathf.Clamp01` мы гарантировано будем возвращать не более 1 (100%). Я сделал

искусственную загрузку длинной в 1.5 секунды + 1 секунда, когда слайдер дойдёт до 100%.

```
1 using System;
2 using System.Collections;
3 using UnityEngine;
4 using UnityEngine.SceneManagement;
5 using UnityEngine.UI;
6
7 public class BootSceneLoader : MonoBehaviour
8 {
9     [Header("UI References")]
10    [SerializeField] public Slider progressBar;
11    [SerializeField] public Image background;
12    [SerializeField] private string startScene = "MainMenu";
13
14    [Header("Settings")]
15    [SerializeField] private float minLoadTime = 1.5f;
16    [SerializeField] private float smoothSpeed = 0.03f;
17
18    [Header("ClearObjects")] [SerializeField]
19    public GameObject[] clearObjects;
20
21    void Start()
22    {
23        StartCoroutine(LoadMainScene());
24    }
25
26    private IEnumerator LoadMainScene()
27    {
28        AsyncOperation op = SceneManager.LoadSceneAsync(startScene, LoadMode.Additive);
29        op.allowSceneActivation = false;
30
31        float timer = 0f;
32        float targetProgress = 0f;
33        float displayedProgress = 0f;
34
35        while (!op.isDone)
36        {
37            timer += Time.deltaTime;
38
39            float sceneProgress = Mathf.Clamp01(op.progress / 0.9f);
40            float timerProgress = Mathf.Clamp01(timer / minLoadTime);
41
42            // Выбираем то, что дальше (грузим минимум minLoadTime)
43            targetProgress = Mathf.Max(sceneProgress, timerProgress);
44
45            // Плавно приближаем отображаемое значение
46            displayedProgress = Mathf.Lerp(displayedProgress, targetProgress, smoothSpeed);
47            progressBar.value = displayedProgress;
48
49            // Проверка на завершение
50            if (sceneProgress >= 1f && timerProgress >= 1f)
51            {
52                yield return new WaitForSeconds(1f);
53                op.allowSceneActivation = true;
54                while (!op.isDone)
55                    yield return null;
56            }
57
58            yield return null;
59        }
60        SceneManager.SetActiveScene(SceneManager.GetSceneByName(startScene));
61        foreach (var clearObject in clearObjects) { Destroy(clearObject); }
62    }
63 }
```

```

63     Scene mainMenuScene = SceneManager.GetSceneByName(startScene);
64     SceneLoader.Instance.SetCurrentScene(mainMenuScene);
65     AudioManager.Instance.PlayMusicType(MusicType.Menu);
66 }
67 }

```

Также укажу, что мы используем здесь `Mathf.Lerp` - известный и простой способ создания плавности. Формула: $A + (B - A) \times t$

Берём точку старта A, точку куда нужно дойти B, и t - процент расстояния от A и B, который нужно пройти за один текущий кадр.

$t = 0.3 \times 0.016 \times 60 \sim 0.288$ (28.8% от оставшегося)

Допустим начало: $0 + (0.66 - 0) \times 0.3 = 0.198$ (20%).

Следующий кадр: $0.19 + (0.66 - 0.19) \times 0.3 = 0.33$ (в этом примере B ещё не успел поменяться)

Это лишь пример, потому что цикл `while` идёт куда чаще (в примере 0.66 означает target за секунду). На деле мы будем прибавлять по крошечным процентам каждую долю секунды, из-за чего получим плавную загрузку, которая местами всё же может быть быстрой.

Если пойти по примерам дальше, то мы увидим, что каждый следующий прыжок значения всё меньше, чем предыдущие. Это называется асимптотическим приближением.

P.s. расписал это больше для себя и для своего углублённого закрепления, чем для самой документации, но почему нет 😊

FadeManager.

i Сцены быстро перещёлкиваются, словно резанные кадры. Иногда это нормально. Например открытию и закрытию меню не всегда нужны какие-то переходы. Но как человек, который много занимался монтажом видео - я люблю когда есть какие-то логичные красивые переходы. Для игры я решил сделать самое простое решение - затемнение экрана и его растемнение.

Напомню, что в момент загрузки уровня я сначала выгружаю одну сцену, а потом загружаю другую. В этот момент камеры в игре нет, а значит у нас чёрный экран. Даже если сделать камеру общей и поместить на `Boot` (персистент) сцену, то экран всё равно будет чёрным. Скорее всего это будет похоже как если бы просто моргнули. Возможно с легким замиранием.

Чтобы это выглядело чуть мягче и логичнее - `Fade` подойдёт в самый раз.

`FadeIn \ FadeInThen` - прокидывают `false` в основной метод.

FadeOut \ FadeOutThen - прокидывают true.

Then приписка показывает о том, что можно дополнительно прокинуть тот самый калбек (порцию методов, которые вызовутся когда событие сработает).

FadeRoutine управляет процессом. Если мы получили false, то значит сейчас у нас включен чёрный квадрат, и нам пора постепенно понизить его alpha (прозрачность). Если прокидываем true - значит прозрачность выключена, а квадрата невидно и нам надо плавно его включить (затухание). Мы также передаём в Lerp внутри While параметры - текущую прозрачность и ожидаемую. И благодаря множеству итераций и самому Lerp делаем плавное затухание и “растухание”.

```
1 using System.Collections;
2 using UnityEngine;
3 using UnityEngine.UI;
4
5 public class FadeManager : Manager<FadeManager>
6 {
7     [Header("References")]
8     [SerializeField] public CanvasGroup fadeCanvas;
9     [SerializeField] public Image fadeImage;
10
11     [Header("Default Settings")] [SerializeField]
12     private float defaultDuration = 1.5f;
13     [SerializeField] private Color defaultColor = Color.black;
14
15     private Coroutine currentFade;
16
17     protected override void Awake()
18     {
19         base.Awake();
20
21         if (fadeCanvas != null)
22         {
23             fadeCanvas.alpha = 0f;
24             fadeCanvas.blocksRaycasts = false;
25         }
26     }
27
28     public static void FadeIn(float duration = -1f, Color? color = null)
29     {
30         if (Instance != null)
31         {
32             Instance.StartFade(false, duration, color);
33         }
34     }
35
36     public static void FadeInThen(System.Action onComplete, float duration, Color? color = null)
37     {
38         if (Instance != null)
39         {
40             Instance.StartFade(false, duration, color, onComplete);
41         }
42     }
43
44     public static void FadeOut(float duration = -1f, Color? color = null)
45     {
46         if (Instance != null)
47         {
48             Instance.StartFade(true, duration, color);
49         }
50     }
51 }
```

```

49     }
50 }
51
52 public static void FadeOutThen(System.Action onComplete, float dur
53 {
54     if (Instance != null)
55         Instance.StartFade(true, duration, color, onComplete);
56 }
57
58 private void StartFade(bool fadeOut, float duration, Color? color
59 {
60     if (currentFade != null)
61     {
62         StopCoroutine(currentFade);
63     }
64
65     currentFade = StartCoroutine(FadeRoutine(fadeOut, duration, co
66 }
67
68 private IEnumerator FadeRoutine(bool fadeOut, float duration, Colo
69 {
70     if (fadeCanvas == null)
71     {
72         yield break;
73     }
74
75     fadeCanvas.blocksRaycasts = true;
76     fadeImage.color = color ?? defaultColor;
77
78     float time = 0f;
79     float start = fadeCanvas.alpha;
80     float end = fadeOut ? 1f : 0f;
81
82     if (duration <= 0)
83     {
84         duration = defaultDuration;
85     }
86
87     while (time < duration)
88     {
89         time += Time.deltaTime;
90         fadeCanvas.alpha = Mathf.Lerp(start, end, time / duration)
91         yield return null;
92     }
93
94     fadeCanvas.alpha = end;
95     if (!fadeOut)
96     {
97         fadeCanvas.blocksRaycasts = false;
98         yield return new WaitForSeconds(1f);
99     }
100
101     onComplete?.Invoke();
102     currentFade = null;
103 }
104 }
105

```

LevelManager.

- В демо версии планируется несколько малоотличающихся уровней в одном стиле. Так как это бесконечный tower defence, то уровень - это просто этаж. Здесь нет градации сложности как в сюжетных tower defence, где каждый раз появляются

новые уровни, новые механики и новые враги. Поэтому (по крайней мере на этапе данного демо) достаточно будет просто выдавать случайный уровень из всех имеющихся.

Чтобы уменьшить повторяемость понадобится хранить информацию о уровнях, что уже выпали до этого.

Я храню список всех возможных уровней. И в случае если их больше пяти (что и планируется), то мы будем хранить очередь из трёх уровней. В момент генерации уровня под следующий этаж просто будем проверять на совпадения с этой очередью. Если нету, то выпавший попадает в очередь, выпихивая вошедшего ранее.

```
1 using System.Collections.Generic;
2 using NUnit.Framework;
3 using UnityEngine;
4
5 public class LevelManager : Manager<LevelManager>
6 {
7     [Header("Levels")]
8     [SerializeField] private List<LevelEntry> levels = new();
9
10    private int currentLevelIndex = -1;
11    private Queue<int> lastPlayed = new(3);
12
13    public int CurrentLevelIndex => currentLevelIndex;
14    public LevelEntry CurrentLevel =>
15        currentLevelIndex >= 0 ? levels[currentLevelIndex] : null;
16
17    public LevelEntry GetRandomLevel()
18    {
19        if (levels.Count == 0)
20        {
21            Debug.LogError("No levels configured!");
22            return null;
23        }
24
25        if (levels.Count < 5)
26        {
27            currentLevelIndex = Random.Range(0, levels.Count);
28            return levels[currentLevelIndex];
29        }
30
31        int index;
32        int safety = 0;
33
34        do
35        {
36            index = Random.Range(0, levels.Count);
37            safety++;
38        }
39        while (lastPlayed.Contains(index) && safety < 20);
40
41        RegisterPlayed(index);
42        currentLevelIndex = index;
43        return levels[index];
44    }
45
46    private void RegisterPlayed(int index)
47    {
48    }
```

```
48         lastPlayed.Enqueue(index);
49
50         if (lastPlayed.Count > 2)
51             lastPlayed.Dequeue();
52     }
53 }
```